11.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-11.1 If we insert the entries $(1,A)$, $(2,B)$, $(3,C)$, $(4,D)$, and $(5,E)$, in this order, into an initially empty binary search tree, what will it look like?

R-11.2 Insert, into an empty binary search tree, entries with keys 30, 40, 24, 58, 48, 26, 11, 13 (in this order). Draw the tree after each insertion.

R-11.3 How many different binary search trees can store the keys $\{1,2,3\}$?

R-11.4 Dr. Amongus claims that the order in which a fixed set of entries is inserted into a binary search tree does not matter—the same tree results every time. Give a small example that proves he is wrong.

R-11.5 Dr. Amongus claims that the order in which a fixed set of entries is inserted into an AVL tree does not matter—the same AVL tree results every time. Give a small example that proves he is wrong.

R-11.6 Our implementation of the TreeMap._subtree_search utility, from Code Fragment 11.4, relies on recursion. For a large unbalanced tree, Python's default limit on recursive depth may be prohibitive. Give an alternative implementation of that method that does not rely on the use of recursion.

R-11.7 Do the trinode restructurings in Figures 11.12 and 11.14 result in single or double rotations?

R-11.8 Draw the AVL tree resulting from the insertion of an entry with key 52 into the AVL tree of Figure 11.14b.

R-11.9 Draw the AVL tree resulting from the removal of the entry with key 62 from the AVL tree of Figure 11.14b.

R-11.10 Explain why performing a rotation in an n-node binary tree when using the array-based representation of Section 8.3.2 takes $\Omega(n)$ time.

R-11.11 Give a schematic figure, in the style of Figure 11.13, showing the heights of subtrees during a deletion operation in an AVL tree that triggers a trinode restructuring for the case in which the two children of the node denoted as y start with equal heights. What is the net effect of the height of the rebalanced subtree due to the deletion operation?

R-11.12 Repeat the previous problem, considering the case in which y's children start with different heights.

R-11.13 The rules for a deletion in an AVL tree specifically require that when the two subtrees of the node denoted as y have equal height, child x should be chosen to be "aligned" with y (so that x and y are both left children or both right children). To better understand this requirement, repeat Exercise R-11.11 assuming we picked the misaligned choice of x. Why might there be a problem in restoring the AVL property with that choice?

R-11.14 Perform the following sequence of operations in an initially empty splay tree and draw the tree after each set of operations.

a. Insert keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.
b. Search for keys 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, in this order.
c. Delete keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.

R-11.15 What does a splay tree look like if its entries are accessed in increasing order by their keys?

R-11.16 Is the search tree of Figure 11.23(a) a $(2,4)$ tree? Why or why not?

R-11.17 An alternative way of performing a split at a node w in a $(2,4)$ tree is to partition w into w' and w'', with w' being a 2-node and w'' a 3-node. Which of the keys k_1, k_2, k_3, or k_4 do we store at w's parent? Why?

R-11.18 Dr. Amongus claims that a $(2,4)$ tree storing a set of entries will always have the same structure, regardless of the order in which the entries are inserted. Show that he is wrong.

R-11.19 Draw four different red-black trees that correspond to the same $(2,4)$ tree.

R-11.20 Consider the set of keys $K = \{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15\}$.

a. Draw a $(2,4)$ tree storing K as its keys using the fewest number of nodes.
b. Draw a $(2,4)$ tree storing K as its keys using the maximum number of nodes.

R-11.21 Consider the sequence of keys $(5,16,22,45,2,10,18,30,50,12,1)$. Draw the result of inserting entries with these keys (in the given order) into

a. An initially empty $(2,4)$ tree.
b. An initially empty red-black tree.

R-11.22 For the following statements about red-black trees, provide a justification for each true statement and a counterexample for each false one.

a. A subtree of a red-black tree is itself a red-black tree.
b. A node that does not have a sibling is red.
c. There is a unique $(2,4)$ tree associated with a given red-black tree.
d. There is a unique red-black tree associated with a given $(2,4)$ tree.

R-11.23 Explain why you would get the same output in an inorder listing of the entries in a binary search tree, T, independent of whether T is maintained to be an AVL tree, splay tree, or red-black tree.

R-11.24 Consider a tree T storing 100,000 entries. What is the worst-case height of T in the following cases?

a. T is a binary search tree.
b. T is an AVL tree.
c. T is a splay tree.
d. T is a $(2,4)$ tree.
e. T is a red-black tree.

R-11.25 Draw an example of a red-black tree that is not an AVL tree.

R-11.26 Let T be a red-black tree and let p be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if p has zero children, the removed node was a red leaf.

R-11.27 Let T be a red-black tree and let p be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if p has one child, the deletion has caused a black deficit at p, except for the case when the one remaining child is a red leaf.

R-11.28 Let T be a red-black tree and let p be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if p has two children, the removed node was black and had one red child.

Creativity

C-11.29 Explain how to use an AVL tree or a red-black tree to sort n comparable elements in $O(n \log n)$ time in the worst case.

C-11.30 Can we use a splay tree to sort n comparable elements in $O(n \log n)$ time in the ***worst case***? Why or why not?

C-11.31 Repeat Exercise C-10.28 for the TreeMap class.

C-11.32 Show that any n-node binary tree can be converted to any other n-node binary tree using $O(n)$ rotations.

C-11.33 For a key k that is not found in binary search tree T, prove that both the greatest key less than k and the least key greater than k lie on the path traced by the search for k.

C-11.34 In Section 11.1.2 we claim that the find_range method of a binary search tree executes in $O(s+h)$ time where s is the number of items found within the range and h is the height of the tree. Our implementation, in Code Fragment 11.6 begins by searching for the starting key, and then repeatedly calling the after method until reaching the end of the range. Each call to after is guaranteed to run in $O(h)$ time. This suggests a weaker $O(sh)$ bound for find_range, since it involves $O(s)$ calls to after. Prove that this implementation achieves the stronger $O(s+h)$ bound.

C-11.35 Describe how to perform an operation remove_range(start, stop) that removes all the items whose keys fall within range(start, stop) in a sorted map that is implemented with a binary search tree T, and show that this method runs in time $O(s+h)$, where s is the number of items removed and h is the height of T.

C-11.36 Repeat the previous problem using an AVL tree, achieving a running time of $O(s \log n)$. Why doesn't the solution to the previous problem trivially result in an $O(s+\log n)$ algorithm for AVL trees?

C-11.37 Suppose we wish to support a new method count_range(start, stop) that determines how many keys of a sorted map fall in the specified range. We could clearly implement this in $O(s+h)$ time by adapting our approach to find_range. Describe how to modify the search tree structure to support $O(h)$ worst-case time for count_range.

C-11.38 If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to maintain support for the new method?

C-11.39 Draw a schematic of an AVL tree such that a single remove operation could require $\Omega(\log n)$ trinode restructurings (or rotations) from a leaf to the root in order to restore the height-balance property.

C-11.40 In our AVL implementation, each node stores the height of its subtree, which is an arbitrarily large integer. The space usage for an AVL tree can be reduced by instead storing the ***balance factor*** of a node, which is defined as the height of its left subtree minus the height of its right subtree. Thus, the balance factor of a node is always equal to -1, 0, or 1, except during an insertion or removal, when it may become ***temporarily*** equal to -2 or $+2$. Reimplement the AVLTreeMap class storing balance factors rather than subtree heights.

C-11.41 If we maintain a reference to the position of the leftmost node of a binary search tree, then operation find_min can be performed in $O(1)$ time. Describe how the implementation of the other map methods need to be modified to maintain a reference to the leftmost position.

C-11.42 If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to accurately maintain the reference to the leftmost position?

C-11.43 Describe a modification to the binary search tree implementation having worst-case $O(1)$-time performance for methods after(p) and before(p) without adversely affecting the asymptotics of any other methods.

C-11.44 If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to maintain the efficiency?

C-11.45 For a standard binary search tree, Table 11.1 claims $O(h)$-time performance for the delete(p) method. Explain why delete(p) would run in $O(1)$ time if given a solution to Exercise C-11.43.

C-11.46 Describe a modification to the binary search tree data structure that would support the following two index-based operations for a sorted map in $O(h)$ time, where h is the height of the tree.

at_index(i): Return the position p of the item at index i of a sorted map.

index_of(p): Return the index i of the item at position p of a sorted map.

C-11.47 Draw a splay tree, T_1, together with the sequence of updates that produced it, and a red-black tree, T_2, on the same set of ten entries, such that a preorder traversal of T_1 would be the same as a preorder traversal of T_2.

C-11.48 Show that the nodes that become temporarily unbalanced in an AVL tree during an insertion may be nonconsecutive on the path from the newly inserted node to the root.

C-11.49 Show that at most one node in an AVL tree becomes temporarily unbalanced after the immediate deletion of a node as part of the standard __delitem__ map operation.

C-11.50 Let T and U be $(2,4)$ trees storing n and m entries, respectively, such that all the entries in T have keys less than the keys of all the entries in U. Describe an $O(\log n + \log m)$-time method for ***joining*** T and U into a single tree that stores all the entries in T and U.

C-11.51 Repeat the previous problem for red-black trees T and U.

C-11.52 Justify Proposition 11.7.

C-11.53 The Boolean indicator used to mark nodes in a red-black tree as being "red" or "black" is not strictly needed when we have distinct keys. Describe a scheme for implementing a red-black tree without adding any extra space to standard binary search tree nodes.

C-11.54 Let T be a red-black tree storing n entries, and let k be the key of an entry in T. Show how to construct from T, in $O(\log n)$ time, two red-black trees T' and T'', such that T' contains all the keys of T less than k, and T'' contains all the keys of T greater than k. This operation destroys T.

C-11.55 Show that the nodes of any AVL tree T can be colored "red" and "black" so that T becomes a red-black tree.

C-11.56 The standard splaying step requires two passes, one downward pass to find the node x to splay, followed by an upward pass to splay the node x. Describe a method for splaying and searching for x in one downward pass. Each substep now requires that you consider the next two nodes in the path down to x, with a possible zig substep performed at the end. Describe how to perform the zig-zig, zig-zag, and zig steps.

C-11.57 Consider a variation of splay trees, called ***half-splay trees***, where splaying a node at depth d stops as soon as the node reaches depth $\lfloor d/2 \rfloor$. Perform an amortized analysis of half-splay trees.

C-11.58 Describe a sequence of accesses to an n-node splay tree T, where n is odd, that results in T consisting of a single chain of nodes such that the path down T alternates between left children and right children.

C-11.59 As a positional structure, our TreeMap implementation has a subtle flaw. A position instance p associated with an key-value pair (k,v) should remain valid as long as that item remains in the map. In particular, that position should be unaffected by calls to insert or delete other items in the collection. Our algorithm for deleting an item from a binary search tree may fail to provide such a guarantee, in particular because of our rule for using the inorder predecessor of a key as a replacement when deleting a key that is located in a node with two children. Given an explicit series of Python commands that demonstrates such a flaw.

C-11.60 How might the TreeMap implementation be changed to avoid the flaw described in the previous problem?

Projects

P-11.61 Perform an experimental study to compare the speed of our AVL tree, splay tree, and red-black tree implementations for various sequences of operations.

P-11.62 Redo the previous exercise, including an implementation of skip lists. (See Exercise P-10.53.)

P-11.63 Implement the Map ADT using a $(2,4)$ tree. (See Section 10.1.1.)

P-11.64 Redo the previous exercise, including all methods of the Sorted Map ADT. (See Section 10.3.)

P-11.65 Redo Exercise P-11.63 providing *positional* support, as we did for binary search trees (Section 11.1.1), so as to include methods first( ), last( ), before(p), after(p), and find_position(k). Each item should have a distinct position in this abstraction, even though several items may be stored at a single node of a tree.

P-11.66 Write a Python class that can take any red-black tree and convert it into its corresponding $(2,4)$ tree and can take any $(2,4)$ tree and convert it into its corresponding red-black tree.

P-11.67 In describing multisets and multimaps in Section 10.5.3, we describe a general approach for adapting a traditional map by storing all duplicates within a secondary container as a value in the map. Give an alternative implementation of a multimap using a binary search tree such that each entry of the map is stored at a distinct node of the tree. With the existence of duplicates, we redefine the search tree property so that all items in the left subtree of a position p with key k have keys that are less than *or equal to* k, while all items in the right subtree of p have keys that are greater than *or equal to* k. Use the public interface given in Code Fragment 10.17.

P-11.68 Prepare an implementation of splay trees that uses top-down splaying as described in Exercise C-11.56. Perform extensive experimental studies to compare its performance to the standard bottom-up splaying implemented in this chapter.

P-11.69 The ***mergeable heap*** ADT is an extension of the priority queue ADT consisting of operations add(k, v), min( ), remove_min( ) and merge(h), where the merge(h) operations performs a union of the mergeable heap h with the present one, incorporating all items into the current one while emptying h. Describe a concrete implementation of the mergeable heap ADT that achieves $O(\log n)$ performance for all its operations, where n denotes the size of the resulting heap for the merge operation.

P-11.70 Write a program that performs a simple n-body simulation, called "Jumping Leprechauns." This simulation involves n leprechauns, numbered 1 to n. It maintains a gold value g_i for each leprechaun i, which begins with each leprechaun starting out with a million dollars worth of gold, that is, $g_i = 1\,000\,000$ for each $i = 1,2,\ldots,n$. In addition, the simulation also maintains, for each leprechaun, i, a place on the horizon, which is represented as a double-precision floating-point number, x_i. In each iteration of the simulation, the simulation processes the leprechauns in order. Processing a leprechaun i during this iteration begins by computing a new place on the horizon for i, which is determined by the assignment

$$x_i = x_i + rg_i,$$

where r is a random floating-point number between -1 and 1. The leprechaun i then steals half the gold from the nearest leprechauns on either side of him and adds this gold to his gold value, g_i. Write a program that can perform a series of iterations in this simulation for a given number, n, of leprechauns. You must maintain the set of horizon positions using a sorted map data structure described in this chapter.

Chapter Notes

Some of the data structures discussed in this chapter are extensively covered by Knuth in his *Sorting and Searching* book [65], and by Mehlhorn in [76]. AVL trees are due to Adel'son-Vel'skii and Landis [2], who invented this class of balanced search trees in 1962. Binary search trees, AVL trees, and hashing are described in Knuth's *Sorting and Searching* [65] book. Average-height analyses for binary search trees can be found in the books by Aho, Hopcroft, and Ullman [6] and Cormen, Leiserson, Rivest and Stein [29]. The handbook by Gonnet and Baeza-Yates [44] contains a number of theoretical and experimental comparisons among map implementations. Aho, Hopcroft, and Ullman [5] discuss $(2,3)$ trees, which are similar to $(2,4)$ trees. Red-black trees were defined by Bayer [10]. Variations and interesting properties of red-black trees are presented in a paper by Guibas and Sedgewick [48]. The reader interested in learning more about different balanced tree data structures is referred to the books by Mehlhorn [76] and Tarjan [95], and the book chapter by Mehlhorn and Tsakalidis [78]. Knuth [65] is excellent additional reading that includes early approaches to balancing trees. Splay trees were invented by Sleator and Tarjan [89] (see also [95]).